

Harnessing the Power of LLMs for Code Smell Detection in Terraform Infrastructure as Code

Quoc-Huy Vo

The Graduate University for Advanced Studies
Tokyo, Japan
huyvq@nii.ac.jp

Ha Dao

Max Planck Institute for Informatics
Saarbrücken, Germany
hadao@mpi-inf.mpg.de

Kensuke Fukuda

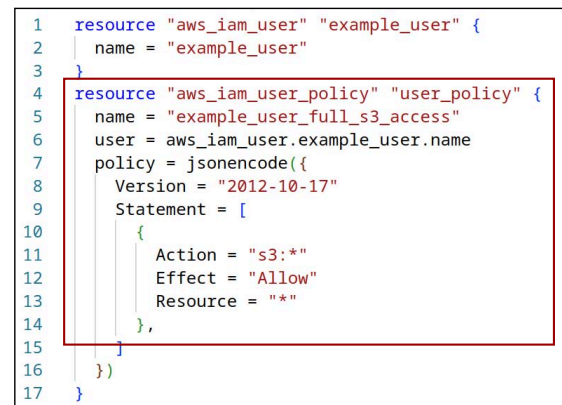
NII/Sokendai
Tokyo, Japan
kensuke@nii.ac.jp

Abstract—Terraform is a widely used Infrastructure as Code (IaC) tool that simplifies cloud resource management through declarative configuration. However, Terraform configurations often exhibit code smells, which can introduce security vulnerabilities, maintainability challenges, and operational inefficiencies. While code smells have been extensively studied in other IaC platforms, research on Terraform remains limited, and existing static analysis tools struggle to detect a broad range of code smells. In this paper, we harness large language models (LLMs) for automated Terraform code smell detection, leveraging their ability to generalize beyond predefined rule-based heuristics. We construct a synthetic benchmark dataset of 42 Terraform configurations covering 14 distinct code smells and evaluate detection performance across traditional linters and LLM-based approaches. Our results show that LLMs significantly outperform static analysis tools, detecting a broader range of code smells, especially logic-related code smells. We extend our analysis to real-world repositories with 71 high-quality Terraform projects from GitHub. Our findings reveal that 84.5% of repositories contain at least one code smell. Notably, some repositories accumulate a high number of unique code smells, with one exhibiting nine distinct issues, underscoring severe quality concerns. Moreover, we find that code smells persist across repositories regardless of popularity, affecting even widely used projects with thousands of stars and forks.

Index Terms—IaC, code smell, LLM

I. INTRODUCTION

Infrastructure as Code (IaC) [1] is a software engineering practice that enables the management and provisioning of infrastructure through code, replacing manual configurations. By defining servers, networking, storage, and other resources in declarative configuration files, IaC allows organizations to achieve consistency, scalability, and automation across both cloud and on-premise environments. This approach also integrates software development practices such as version control, automated testing, and continuous deployment, improving maintainability and security. To implement IaC, various management platforms have been developed, including Puppet [2], Chef [3], TOSCA [4], and Terraform [5]. These platforms provide different automation mechanisms to simplify infrastructure provisioning and configuration management. Among them, Terraform has emerged as a widely adopted IaC tool due to its declarative configuration language, multi-cloud support, and state management capabilities. Terraform enables modular, reusable, and scalable infrastructure definitions, making it a preferred choice for cloud-native deployments [6].



```

1 resource "aws_iam_user" "example_user" {
2   name = "example_user"
3 }
4 resource "aws_iam_user_policy" "user_policy" {
5   name = "example_user_full_s3_access"
6   user = aws_iam_user.example_user.name
7   policy = jsonencode({
8     Version = "2012-10-17"
9     Statement = [
10      {
11        Action = "s3:*"
12        Effect = "Allow"
13        Resource = "*"
14      },
15    ]
16  })
17 }

```

Fig. 1. Terraform code smells “admin by default” example.

Despite Terraform’s benefits, there has been limited research exploring code smells within its configuration files, unlike Puppet, Chef, and TOSCA, where prior studies have extensively documented such issues [7]–[9]. Code smells—which indicate suboptimal design choices or security risks—have been widely studied across various software engineering domains, including database management, logging, and continuous integration [10]. In the context of Terraform, adhering to best practices is crucial, as code smells can introduce misconfigurations, security vulnerabilities, and operational inefficiencies. However, without automated detection mechanisms, developers may unknowingly introduce these issues, impacting the reliability, maintainability, and security of the managed infrastructure. For example, consider a Terraform configuration that exhibits the “admin by default” code smell in an Amazon Web Services Identity and Access Management (AWS IAM) policy (see Figure 1). This configuration grants the IAM user `example_user` unrestricted access to all Simple Storage Service (S3) operations (`s3:*`) across all resources (`Resource = "*"`), violating the principle of least privilege [11] and posing serious security risks: (1) compromised credentials could grant attackers full access to all S3 buckets, leading to data breaches or data loss; (2) broad permissions obscure actual resource usage, complicating security

audits; and (3) maintenance becomes challenging without clear documentation of necessary permissions. While simplified for clarity, this issue is critical in production environments. As Fowler’s refactoring book, refactoring may seem unnecessary in isolated cases but becomes essential as systems scale [12]. In large-scale deployments, permissive IAM policies can propagate across multiple users and services, amplifying security risks. Without systematic refactoring, tracking and maintaining permissions becomes increasingly complex, jeopardizing security and manageability.

These challenges in security policy management have prompted exploration of advanced automation techniques to assist with identification, analysis, and refactoring of problematic security configurations. In this context, artificial intelligence—particularly large language models—offers promising capabilities for augmenting traditional security analysis approaches.

The rapid advancement of large language models (LLMs) has been transforming the fields of privacy and cybersecurity [13], [14], with several studies investigating their potential in static analysis and vulnerability detection [15]–[19]. Unlike traditional rule-based analysis tools, e.g., *Tfsec* [20], *Tflint* [21], that rely on manually curated policies, LLMs can generalize across diverse code structures, infer best practices from large-scale code corpora, and detect subtle yet critical code smells that may otherwise go unnoticed. Whereas existing tools enforce predefined best practices and security guidelines, they often fall short in detecting complex issues, such as improper privilege escalation, misaligned resource dependencies, or redundant configurations that degrade maintainability. LLMs, with their contextual understanding, offer a promising avenue for identifying such issues by interpreting code semantics and suggesting improvements beyond static rule enforcement.

In this paper, we present an empirical evaluation of LLMs’ capabilities in detecting code smells in Terraform configurations. Our contributions are summarized as follows:

- We evaluate the performance of LLMs and traditional linters in detecting 14 code smells in Terraform using synthetic data, revealing that LLMs significantly outperform static analysis tools by identifying a broader range of security and logic-related issues that traditional linters fail to capture. (§ III).
- We collect Terraform repositories from GitHub and apply a rigorous filtering process, selecting 71 high-quality projects based on criteria such as community interest, project maturity, and ongoing maintenance to ensure a meaningful analysis of code smells (§ IV-A). We then develop a hybrid detection strategy that combines traditional linters with LLMs to characterize code smells in the collected public GitHub repositories (§ IV-B). This approach prioritizes linters for rule-based static analysis while leveraging LLMs to detect specific Terraform code smells that linters fail to identify..
- We then characterize these code smells in real-world open-source Terraform repositories. Our analysis shows

that 84.5% of Terraform repositories contain at least one code smell, with access control issues such as *unrestricted_ip_address* (81.83%) and *admin_by_default* (5.72%) being the most frequent, highlighting the security risks posed by overly permissive configurations. Additionally, a subset of Terraform repositories contains a large number of code smells, with one repository exhibiting up to nine unique smells, indicating substantial technical debt in these codebases (§ IV-C).

- When analyzing the popularity of Terraform GitHub repositories, we discover that even some repositories with a large number of subscribers, stars, and forks, also contain code smells that significantly influence code quality issues. Our results also point out the broad coverage of code smells in the repositories containing Terraform code (§ IV-D).

II. BACKGROUND AND RELATED WORK

A. Background

1) *IaC* : IaC is a pivotal DevOps methodology at the forefront of software deployment onto cloud platforms. IaC relies on source code files rather than manual configuration to manage the infrastructure of a software system. The concept of IaC was introduced by Chad Fowler in a 2013 blog post [22]. The term “as code” in IaC reflects the use of established software engineering practices, including code review and version control, for managing IaC scripts [23], [24]. To provision infrastructure automatically, developers write configurations using a syntax similar to that of software source code. IaC scripts are written in domain-specific languages [25]. Organizations adopting DevOps practices commonly use commercial tools to implement IaC [24]–[26]. This approach applies software development practices to cloud infrastructure, enabling debugging, version control, and automation [27]. IaC allows for the programmatic management of cloud resources to ensure scalability and consistency. Notable tools include Puppet [2], Chef [3], TOSCA [4], and Terraform [5], which provide a declarative framework for defining and managing infrastructure.

2) *Terraform*: Terraform is one of the most famous IaC management tools with large community support [6]. Terraform has two logical parts [6]: core and plugins. While Terraform core (e.g., CLI in developer’s machine) communicates with user-defined manifests and Terraform plugins (typically Terraform registry ¹), each plugin takes a role like a middleware to communicate which specific service or platform provider (e.g., Docker ² or Google Cloud Platform ³), or provisioned (e.g., bash ⁴).

3) *Code smells*: Beck and Fowler [12] introduced the concept of code smells, which indicate potential design flaws without causing immediate runtime errors. A code smell is

¹<https://registry.terraform.io/>

²<https://www.docker.com/>

³<https://cloud.google.com/>

⁴<https://www.gnu.org/software/bash/>

a surface symptom suggesting deeper issues in design or implementation. While not bugs, code smells highlight areas for improvement, aiding proactive refactoring to enhance code quality and maintainability [12]. Similar to traditional software development, IaC faces *configuration code smells*, which signal weaknesses in scripts that may impact the maintainability, reliability, scalability, or security of cloud infrastructure.

B. Related work

1) *Code smell detection in IaC*: Recognizing the potential impact of configuration code smells, research efforts have explored methods for their early detection and prevention in IaC [7]–[9], [28]–[31]. These studies focus on identifying and mitigating such smells, ultimately aiming to enhance the quality and reliability of IaC codes using Chef, Puppet, Ansible, and TOSCA.

There are some studies on characterizing code smells on GitHub repositories. Rahman et al. [32] present another comprehensive analysis of security misconfigurations in Kubernetes manifests to enhance cluster security for practitioners by examining 2,039 manifests from open-source repositories. Wu et al. [33] provide a comprehensive empirical study on characterizing the occurrence of Dockerfile smells in open-source software, shedding light on the prevalence, distribution, co-occurrence, and correlation of Dockerfile smells with project characteristics in GitHub repositories. Saavedra et al. [34], [35] made strides in security smell detection by introducing GLITCH, a technology-agnostic framework that effectively identifies security smells across Ansible, Chef, Puppet, and Terraform scripts, validating their approach through a large-scale study of nearly 200,000 IaC scripts. More recently, Begoug et al. [36] advanced the field by investigating defect prediction at the block level in IaC, specifically focusing on Terraform files. They demonstrated that machine learning techniques can effectively predict defects through an evaluation of 19 open-source Terraform projects.

However, despite these contributions, there remains a significant gap in the literature regarding the systematic characterization of code smells in Terraform-based IaC across large-scale GitHub repositories. In response to these challenges and research gaps, several tools have emerged to address quality and security concerns in Terraform code, with *Tflint* and *Tfsec* being prominent examples. *Tflint*, developed by Terraform creators HashiCorp, focuses on detecting possible errors and enforcing best practices in Terraform code, particularly for provider-specific rules and basic syntax validation [21]. Similarly, *Tfsec* specializes in security-focused static analysis of Terraform code, identifying potential security vulnerabilities and misconfigurations [20]. These tools have gained widespread adoption in the DevOps community for their ability to catch common issues early in the development cycle. However, their primary focus remains on provider-specific rules, syntax checking, and security-centric analysis rather than comprehensive code smell detection.

2) *LLMs in Static Analysis and Code Validation*: Several studies have explored the potential of LLMs in static analy-

sis and vulnerability detection [15]–[19]. Ahmad et al. [15] curated a corpus of hardware security bugs and developed a framework to evaluate LLMs, showing that an ensemble of LLMs outperforms a state-of-the-art repair tool by successfully fixing all fifteen benchmarks. Kwon et al. [16] explored the applicability of LLM-based Automated Program Repair on Ansible, evaluating ChatGPT and Bard on 58 Ansible script revision cases from open-source software. Their findings indicate promising potential, with LLMs generating helpful responses in 70% of the sampled cases. Li et al. [17] explored how LLMs can refine static analysis by reducing false positives in bug-finding tools, with GPT-4 successfully pruning 16 out of 20 cases, demonstrating their potential for improving program analysis. Liu et al. [18] proposed LATTE, the first LLM-powered static binary taint analysis, and demonstrated its scalability and potential for advancing binary vulnerability analysis. Yu et al. [19] conducted an empirical study on using LLMs for security code review, evaluating six models under five prompts against state-of-the-art static analysis tools, showing that GPT-4 outperforms these tools.

Recent studies have leveraged LLMs for misconfiguration detection, improving validation accuracy and reducing reliance on manual rules. Lian et al. [37] introduced an LLM-based validation framework using prompt engineering and few-shot learning. Malul et al. [38] proposed GenKubeSec for Kubernetes misconfigurations, achieving expert-validated accuracy. Ye et al. [39] developed LLMSecConfig, integrating static analysis with LLMs to repair security misconfigurations in container orchestrators with a 94% success rate. Our work extends this line of research by applying LLM-based code smell detection to Terraform. We address 14 code smells across security, maintainability, performance, and portability, combining LLM analysis with static tools for precise detection and remediation.

III. METHODOLOGY

In this section, we describe the construction of synthetic data for evaluating LLM-based detection and traditional linters in identifying Terraform code smells. The goal is to systematically assess the strengths and limitations of both approaches using a controlled dataset that captures common code smells across various Terraform configurations.

A. Synthetic Data Generation

We survey code smells documented in previous studies on IaC management platforms [7]–[9], identifying those that align with Terraform’s configuration syntax. Since some code smells are specific to certain programming languages, we assess their applicability within the Terraform platform and select 14 relevant code smells (see § B in the Appendix). Based on this selection, we construct a benchmark dataset for evaluating Terraform code smell detection, comprising 14 distinct code smells, each with three variations, resulting in 42 synthetic Terraform configuration files that replicate code smells commonly found in Terraform-based IaC deployments.

B. Code Smell Detection Approaches

We then employ four established static analysis linters: *Tflint*, *Tfsec*, *Checkov*, and *Terrascan* to analyze our synthetic dataset. The tool selection was based on three criteria: (1) active maintenance with regular updates within the last six months, (2) community adoption rates exceeding 1,000 GitHub stars, and (3) complementary detection capabilities across different types of infrastructure issues. Each tool provides specialized analysis capabilities: *Tflint* (0.45.0) focuses on potential errors and style violations, *Tfsec* (1.28.1) specializes in security compliance, *Checkov* (2.3.187) emphasizes cloud infrastructure misconfigurations, and *Terrascan* (1.18.3) covers general security vulnerabilities.

Beyond static analysis tools, we leverage LLM-based detection using OpenAI's GPT-3.5-turbo (gpt-3.5-turbo), GPT-4 (gpt-4-0613) and GPT-4o (gpt-4o-2024-11-20), selected for their advanced capabilities in code comprehension, natural language processing, code understanding, and explanation [40]–[43]. We use zero-shot learning to eliminate reliance on external sample configurations, which requires no prior examples, making it a widely used optimization technique [44], [45]. While few-shot learning improves effectiveness by leveraging labeled samples during inference [37], [46], [47], its performance depends on sample quality and selection. In contrast, zero-shot learning avoids the cost and effort of sample collection, making it the preferred approach for our framework. Our method leverages OpenAI's GPT-3.5-turbo, GPT-4, and GPT-4o without requiring task-specific fine-tuning, allowing for scalable and flexible analysis. The prompt is designed with the following key components:

- **Explicit Task Definition:** The prompt instructs the model to analyze Terraform code smells. To ensure precision and relevance, the model is restricted to detecting only a predefined set of 14 code smells. This prevents misclassification and reduces false positives.
- **Structured Output Format:** The model is required to return results in a predefined JSON format with the following fields:
 - `file_path`: The location of the analyzed file.
 - `code_smell_type`: One of the predefined code smells.
 - `description`: A concise explanation of the identified issue.
 - `from_line_number` and `to_line_number`: The affected line numbers.
- **Model Constraints:** The prompt explicitly directs the model to return JSON-formatted responses, ensuring compliance with the structured schema. Non-conforming responses are discarded.
- **System and User Roles in Prompting:** We structure the interaction with GPT models using distinct roles to improve response consistency. The system message explicitly defines the model's role as a *code security expert* analyzing Terraform code, ensuring a focused analysis. The user message contains the structured prompt,

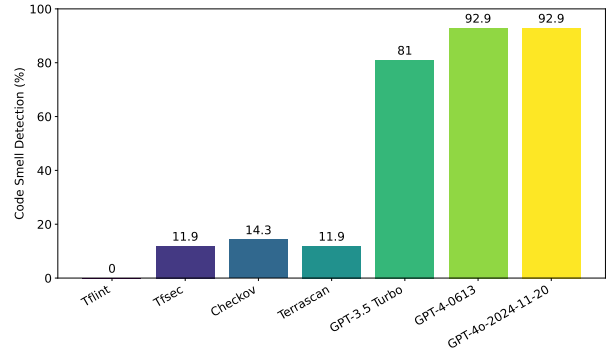


Fig. 2. Code smell detection performance across synthetic data by various traditional linters and LLM models.

including the predefined code smell list and expected response format, directing the model to adhere strictly to JSON-formatted output.

- **Implementation Workflow:** Terraform files from GitHub repositories are processed and submitted to GPT models for analysis. The system validates the model's output to confirm adherence to the predefined format and the correctness of the detected code smells.

Each linter and model is run independently on the synthetic dataset, and the outputs are validated against the ground truth labels.

C. Validation

We present the ability of traditional linters and LLMs models to detect code smells within our synthetic Terraform dataset in Figure 2. We observe that traditional linters exhibit limited detection capabilities, which identify only a small fraction of the predefined code smells. Static analysis tools rely on rule-based heuristics that may not generalize well to diverse Terraform code smells. Furthermore, we hypothesize that this gap in detection may stem from a disconnect between research and practice, as only a small subset of the code smells documented in prior studies is covered by these widely used linters. This lack of alignment indicates that many security-relevant code smells identified in academic research are not yet incorporated into commonly adopted tooling, highlighting a potential opportunity for improving Terraform security analysis through closer collaboration between researchers and tool developers. Conversely, LLM-based models achieve significantly higher detection rates, demonstrating substantial improvements over static analysis tools. Notably, GPT-4 and GPT-4o detect up to 92.9% of code smells, outperforming both traditional linters and other LLM variants. This result suggests that LLMs can generalize beyond rule-based approaches, potentially identifying patterns and code smells that static analysis tools fail to capture.

We further investigate the detection coverage of traditional linters and LLM models for each code smell in Figure 3. We observe a clear disparity between these two categories, with

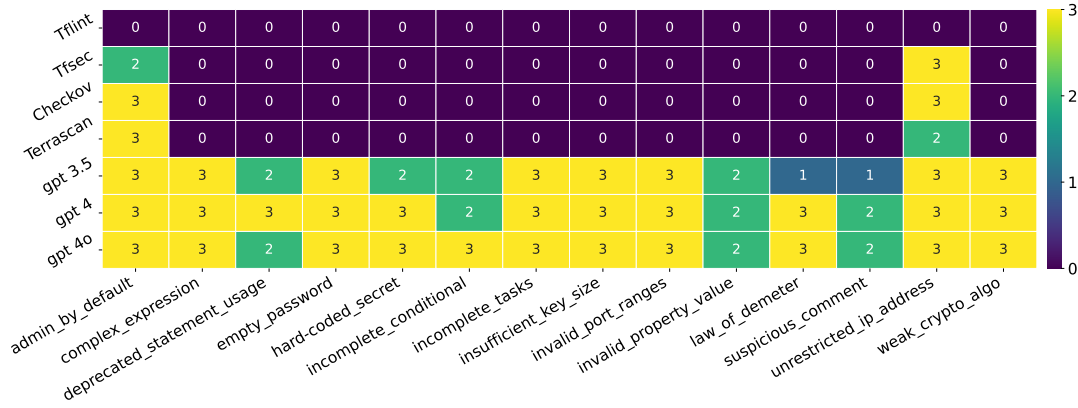


Fig. 3. Detection coverage of code smells in Terraform by traditional linters and LLM models.

LLM-based approaches demonstrating significantly broader coverage across various Terraform code smells. Traditional linters show highly selective detection capabilities, effectively identifying a few security-related smells such as *unrestricted_ip_address* and *admin_by_default* while failing to recognize more nuanced issues related to logic and maintainability. In contrast, LLMs consistently demonstrate a higher detection rate across all predefined Terraform code smells. These models successfully capture complex patterns, including logic-based issues like *incomplete_tasks* and *complex_expression*, which traditional static analysis tools fail to detect. The variation in coverage among LLMs suggests that more advanced iterations, such as GPT-4o, offer improved sensitivity to a wider range of code smells, particularly those related to maintainability and best practices. Additionally, we observe that GPT-based models can identify code smells that were previously undocumented in linter rule sets, further reinforcing the potential of AI-driven approaches in augmenting Terraform security analysis. This disparity underscores the limitations of existing linters in handling diverse Terraform code smells beyond predefined rule sets. While traditional tools remain valuable for enforcing well-established security practices, their rule-based nature constrains their ability to detect emerging or logic-driven code smells. The broader coverage observed in LLM-based approaches suggests that integrating machine learning techniques into Terraform static analysis could bridge this gap, improving the detection of both security and maintainability-related issues. Our findings highlight the need for further exploration into hybrid methodologies that combine the precision of rule-based static analysis with the adaptability of AI-driven models to enhance Terraform security and maintainability.

We also note that the high detection rate of LLMs raises concerns about potential false positives. To address this, we conduct further validation to assess the reliability of LLM-based detection in practical Terraform deployments. Our analysis confirms that the detection quality of GPT-3.5 Turbo is acceptable, ensuring that LLMs can effectively identify

code smells without introducing excessive misclassifications. Although GPT-3.5 Turbo exhibits a manageable false positive rate (15.3%), in production settings or automated workflows, lower false positive rates may be required to prevent unnecessary or risky code changes. We recommend that developers manually verify suggestions before applying changes, particularly in safety-critical infrastructure. In contrast, traditional static analysis tools in our evaluation showed a 0% false positive rate, likely due to their limited detection scope and strict rule-based logic (see § A in the Appendix).

IV. CHARACTERIZING CODE SMELLS IN GITHUB REPOS

A. Data Collection and Project Filtering

First, we retrieve data from the GitHub open source dataset on Google BigQuery [48]. Since the “.tf” file extension is classified under the “HCL” group in GitHub’s language qualifier [49], we filter repositories where the language is set to “HCL”. This query returns 3,068 GitHub repositories, along with their repository names (“*repo_name*”). Using these repository names, we construct GitHub Clone URLs in the format: “<https://github.com/>” + “*repo_name*” + “.git”. Of the 3,068 repositories, we successfully clone 2,742 repositories on our server (232 GB), with the remaining repositories being deleted or set to private before retrieval.

In addition, to maintain data quality and ensure meaningful analysis, we implemented a comprehensive filtering strategy to exclude low-quality and toy repositories from our dataset. Drawing from established research practices [50]–[53], we defined five key criteria for repository inclusion: (1) more than 10 repository stars to indicate community interest; (2) non-forked status to focus on original projects; (3) a substantial commit history exceeding 200 commits demonstrating project maturity; (4) an active contributor base of more than two contributors to ensure collaborative development; and (5) recent maintenance activity with commits within the last year to confirm ongoing project vitality. These stringent filtering criteria were systematically applied through our analysis pipeline to create a refined dataset comprising only substantive, actively

maintained software projects. After applying these criteria, our final dataset consisted of 71 high-quality projects from GitHub based on indicators of popularity and maintenance quality, including the number of stars, contributors, and recent commit activity. These criteria were chosen to ensure that the repositories analyzed would have sufficient development history, active maintenance, and consistent use of development practices, enabling a reliable analysis of code quality and smells.

B. Code Smell Detection

To systematically detect Terraform code smells in GitHub repositories, we adopt a structured prioritization approach that selects the most suitable tool based on effectiveness and reliability (see § III-C). Our methodology first relies on linters, as they provide rule-based static analysis and enforce Terraform security best practices. When multiple linters detect the same code smell, we prioritize them based on their specialization, selecting *Checkov* over *Terrascan* for privilege escalation risks, e.g., *admin_by_default* and *Tfsec* over *Checkov* for network security misconfiguration, e.g., *unrestricted_ip_address*. If no linter detects all examples of a code smell, we leverage LLMs, prioritizing GPT-3.5 Turbo due to its strong code comprehension and structural pattern recognition. GPT-3.5 Turbo effectively identifies 14 code smells while maintaining an acceptable false positive rate for practical use (see § A in the Appendix). This approach ensures a balanced and scalable detection strategy, integrating the precision of linters with the flexibility of LLMs to enhance Terraform misconfiguration detection in GitHub repositories.

C. Code Smells in Terraform Characterization

Here, we characterize code smells on Terraform Github repositories.

Overall, 84.5% of Terraform GitHub repositories contain at least one Terraform code smell. Terraform code smells are prevalent in GitHub repositories, even after applying our strict filtering criteria to exclude “toy” projects (see § IV-A). This finding highlights the widespread presence of code smells in real-world Terraform code, emphasizing the need for improved detection and mitigation strategies.

We then provide a quantitative analysis of 13 code smells across 60 public Terraform repositories (each repository has at least one code smell), as shown in Table I. Although we searched for 14 code smell types, we found zero instances of *weak_crypto_algorithms*, leaving 13 code smells for our analysis. Our findings reveal that access control-related code smells, such as *unrestricted_ip_address* (81.83% of code smells, found in 85.0% of repositories) and *admin_by_default* (5.72% of code smells, found in 20.0% of repositories), are particularly widespread. This suggests that many Terraform configurations adopt overly permissive security settings, either by allowing unrestricted IP access or assigning administrative privileges by default. Such practices increase the risk of unauthorized access and privilege escalation, making infrastructure deployments

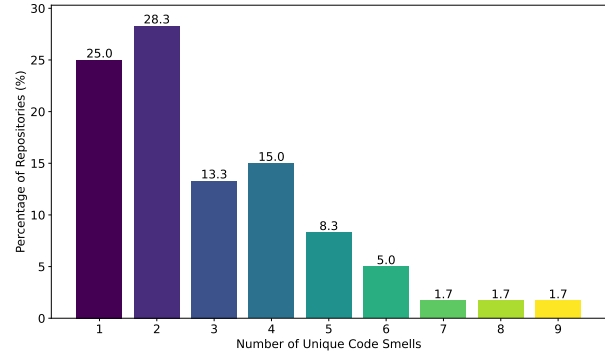


Fig. 4. Distribution of repositories based on the number of unique code smells associated with them. The percentage was calculated by the total number of 60 Terraform GitHub repositories containing code smells.

more susceptible to security breaches. Beyond access control, we observe logic-related code smells, including *incomplete_tasks* (3.09%) and *suspicious_comment* (1.95%), which indicate infrastructure definitions that are either incomplete or inconsistent, making Terraform scripts harder to maintain and verify in deployments.

Figure 4 shows the distribution of Terraform repositories based on the number of unique code smells detected. The majority of repositories exhibit a small number of unique code smells, with the distribution gradually decreasing as the number of detected smells increases. However, a subset of repositories contains a significantly higher number of unique code smells, i.e., indicating a concentration of code smells in certain projects. While code smells are prevalent across Terraform repositories, their severity varies, with some repositories exhibiting a higher accumulation of issues.

We further analyze the distribution of code smells across repositories for the top 10 programming languages used in conjunction with Terraform. As shown in Table II, Go, HCL, Jinja, TypeScript, Python, Java, Rust, Jupyter Notebook, C, and Groovy emerge as the languages most frequently associated with repositories containing code smells. Go and HCL repositories exhibit the highest number of Terraform code smells, with 1,882 and 706 instances, respectively, which corresponds to their prevalence in the dataset as the languages with the most total repositories. Following these, Jinja, TypeScript, and Python repositories also contain significant numbers of Terraform code smells relative to their total repository counts. This pattern suggests that projects using these popular languages may involve more complex or legacy code structures, potentially leading to a higher incidence of code smells.

D. How Do Popular Terraform Repositories Contain Code Smells?

We analyze the relationship between popular repositories and Terraform code smells. To this end, we inspect the number of stars, subscribers, and forks repositories receive as

TABLE I
DISTRIBUTION OF CODE SMELL CATEGORIES ACROSS REPOSITORIES.

#	Code Smell	Tools	Number of code smells / Percentage	Number of repos / Percentage
1	unrestricted_ip_address	<i>Tfsec</i>	2517 / 81.83%	51 / 85.0%
2	admin_by_default	<i>Checkov</i>	176 / 5.72%	12 / 20.0%
3	incomplete_tasks	GPT-3.5 Turbo	95 / 3.09%	31 / 51.67%
4	incomplete_conditional	GPT-3.5 Turbo	64 / 2.08%	19 / 31.67%
5	suspicious_comment	GPT-3.5 Turbo	60 / 1.95%	21 / 35.0%
6	deprecated_statement_usage	GPT-3.5 Turbo	49 / 1.59%	6 / 10.0%
7	invalid_property_value	GPT-3.5 Turbo	36 / 1.17%	1 / 1.67%
8	empty_password	GPT-3.5 Turbo	31 / 1.01%	15 / 25.0%
9	invalid_port_ranges	GPT-3.5 Turbo	19 / 0.62%	2 / 3.33%
10	law_of_demeter	GPT-3.5 Turbo	14 / 0.46%	7 / 11.67%
11	complex_expression	GPT-3.5 Turbo	12 / 0.39%	8 / 13.33%
12	insufficient_key_size	GPT-3.5 Turbo	2 / 0.07%	2 / 3.33%
13	hard_coded_secret	GPT-3.5 Turbo	1 / 0.03%	1 / 1.67%
Total			3076 / 100%	60 / 100%

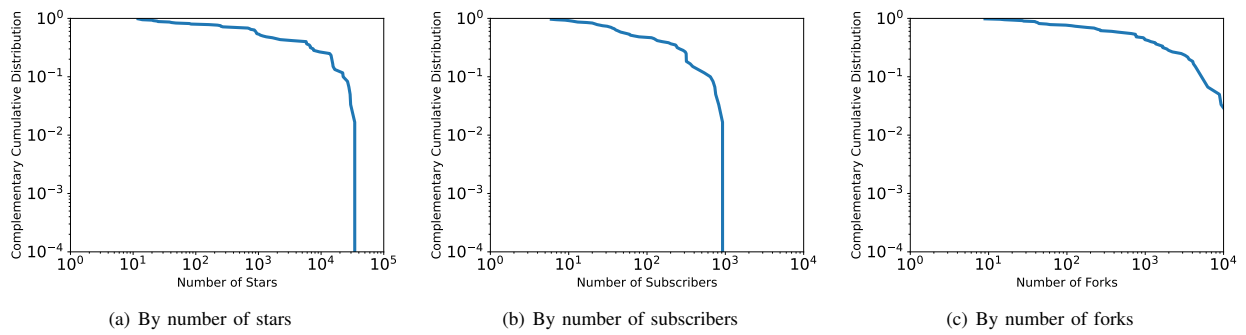


Fig. 5. Distribution of Terraform smell repositories on popular GitHub repositories.

a basic proxy of their popularity. Specifically, we analyze the distribution of repositories containing Terraform code smells by the number of stars, subscribers, and forks.

Figure 5 presents the complementary cumulative distribution functions (CCDFs) of repositories exhibiting Terraform code smells across these three popularity indicators. We observe that code smells appear across all levels of popularity, from repositories with minimal engagement to those with substantial community adoption. While less popular repositories often lack sufficient review processes, making them more prone to code quality issues, even widely used

repositories with over 10,000 stars continue to exhibit code smells, suggesting that popularity alone does not guarantee adherence to best practices in Infrastructure as Code. Similarly, the presence of code smells in frequently forked repositories raises concerns about the propagation of flawed infrastructure configurations across projects, potentially amplifying security and maintainability risks within the ecosystem. Furthermore, the CCDF for subscribers highlights that even repositories with significant community monitoring continue to contain code smells, indicating that either these issues are deprioritized by maintainers or require significant effort to address. These findings demonstrate that Terraform code smells are not confined to obscure or inactive projects but are prevalent even in highly visible and widely adopted repositories, emphasizing the need for systematic efforts to improve code quality in Terraform infrastructure.

TABLE II
COVERAGE OF CODE SMELLS FOR TOP-10 PROGRAMMING LANGUAGES.

#	Language	# Repositories	# Code smells
1	Go	21	1,882
2	HCL	8	706
3	Jinja	4	113
4	TypeScript	4	108
5	Python	7	92
6	Java	6	89
7	Rust	1	24
8	Jupyter Notebook	2	17
9	C	1	12
10	Groovy	1	11

V. DISCUSSION

A. Limitations

Firstly, while the synthetic dataset provides a controlled environment for evaluating code smell detection, it may not fully capture the complexity and variability of real-world Terraform projects. However, its structured nature ensures a reproducible and systematic evaluation of detection approaches,

enabling precise comparisons between different methods. Furthermore, our findings from real-world repositories reinforce the relevance of the code smells identified in synthetic data, suggesting that key detection patterns remain applicable across different contexts. Secondly, the dataset consists of public repositories, which may not fully represent Terraform usage in private or enterprise settings. Nevertheless, public repositories offer a broad and diverse sample of Terraform configurations, reflecting widely adopted infrastructure practices. Future research could improve the generalizability of findings by incorporating private datasets, where feasible, and engaging with industry practitioners to analyze proprietary Terraform deployments. Finally, this study focuses on a predefined set of 14 Terraform code smells, selected based on their prevalence in prior research and security best practices. While this approach may not capture the full range of code quality issues in Terraform projects, it provides a structured foundation for systematic detection. Expanding this taxonomy in future work will allow for a more comprehensive assessment of Terraform code quality while maintaining the methodological rigor established in this study.

B. Enhancing Code Smell Detection with LLMs and Linters

To mitigate the prevalence of Terraform code smells, developers can adopt a multi-layered approach that combines proactive detection, structured validation, and best practices enforcement. Traditional linters provide rule-based detection for well-defined security issues, but their reliance on static heuristics often results in limited coverage of logic-related code smells. By integrating LLM-based detection tools alongside linters, developers can enhance identification accuracy and capture a broader range of misconfigurations that conventional static analysis tools overlook. LLMs, with their ability to recognize contextual patterns and infer infrastructure intent, can complement linters by detecting incomplete tasks, suspicious comments, and configuration inconsistencies that may impact security and maintainability. Incorporating these tools into continuous integration (CI) pipelines and pre-deployment validation processes ensures that misconfigurations are detected and addressed early, reducing technical debt and preventing insecure infrastructure deployments.

C. Outperformance of GPT-3.5 Turbo

Regarding the best performance of GPT-3.5 Turbo (see § A), we hypothesize that although GPT-3.5 Turbo is smaller than GPT-4 and GPT-4o, it may have been optimized through fine-tuning or instruction-tuning specifically for code understanding related tasks, leading to better alignment with our defined code smell detection criteria. Conversely, GPT-4 and GPT-4o possess a much broader and more diverse knowledge base, which is beneficial for general reasoning tasks, may cause then to overcomplicate the problems. This overcomplication can increase hallucinations and subsequently lead to higher FP rates. Further controlled analysis would be valuable to confirm this hypothesis.

D. Extending datasets and code smell coverage

Our study currently uses a predefined set of 14 code smells, each represented through descriptive prompts. While this approach has proven effective, extending the set would enable broader and more comprehensive detection capabilities. The most straightforward method is to expand the prompt template, but this is constrained by the context window limitations of LLMs. A more scalable alternative is to construct a knowledge base of code smells and integrate it with the LLM via retrieval-augmented generation (RAG), a direction we plan to explore in future work.

To address limitations in both coverage and realism, we also plan to expand our synthetic dataset by incorporating a wider variety of real-world code smells. This includes introducing more complex and logic-intensive structures (e.g., nested modules, cross-provider interactions) and increasing variability within each smell type. We are further exploring the use of LLMs to generate synthetic examples, which can improve diversity while reducing manual effort.

Finally, we aim to broaden our repository selection beyond high-quality and widely-used projects. By analyzing smaller and less mature repositories, we hope to understand whether code smell patterns differ across project types and development maturity levels.

VI. CONCLUSION

In this paper, we investigated Terraform code smells and evaluated the effectiveness of large language models (LLMs) in detecting them. Our study showed that LLMs significantly outperform traditional linters, particularly in identifying logic-related code smells such as “suspicious comment” or “complex expression”, which static tools often overlook due to their reliance on predefined rule sets. However, while LLMs show promising complementary capabilities, their advantage is more pronounced in complex or nuanced scenarios (as reflected in the synthetic dataset) rather than in common, well-known issues found in public datasets. Using a synthetic dataset of 42 Terraform configurations covering 14 code smells, we demonstrated the broader detection capabilities of LLMs compared to rule-based static analysis tools. Extending our analysis to real-world repositories, we examined 71 high-quality Terraform projects from GitHub and found that 84.5% contain at least one code smell, highlighting security risks in widely used configurations. Moreover, we observed that code smells persist across repositories regardless of popularity, with some projects accumulating multiple issues, emphasizing the need for better detection and mitigation strategies. Our findings highlight the limitations of existing Terraform analysis tools and demonstrate the potential of LLMs in improving IaC security and maintainability.

ACKNOWLEDGMENTS:

This work was partially supported by JST ASPIRE (JPM-JAP2325) and the CIS internship program at the Max Planck Institute for Informatics.

REFERENCES

- [1] K. Morris, *Infrastructure as code: managing servers in the cloud*. O'Reilly Media, Inc., 2016.
- [2] "Puppet infrastructure and it automation at scale," accessed February 1, 2025. [Online]. Available: <https://www.puppet.com/>
- [3] "Configuration management system software - chef infra," accessed February 1, 2025. [Online]. Available: <https://www.chef.io/products/chef-infra>
- [4] "Opentasca," accessed February 1, 2025. [Online]. Available: <https://www.opentasca.org/>
- [5] "Terraform by hashicorp," accessed February 1, 2025. [Online]. Available: <https://www.terraform.io/>
- [6] Y. Brikman, *Terraform: Up and Running*. O'Reilly Media, Inc., 2022.
- [7] T. Sharma, M. Frangkoulis, and D. Spinellis, "Does your configuration code smell?" in *Proc. MSR*, 2016, pp. 189–200.
- [8] J. Schwarz, A. Steffens, and H. Lichter, "Code smells in infrastructure as code," in *Proc. IEEE QUATIC*, 2018, pp. 220–228.
- [9] I. Kumara, Z. Vasileiou, G. Meditskos, D. A. Tamburri, W.-J. Van Den Heuvel, A. Karakostas, S. Vrochidis, and I. Kompatsiaris, "Towards semantic detection of smells in cloud infrastructure code," in *Proc. ACM WISM*, 2020, pp. 63–67.
- [10] T. Sharma and D. Spinellis, "A survey on software smells," *Journal of Systems and Software*, pp. 158–173, 2018.
- [11] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proc. IEEE*, pp. 1278–1308, 1975.
- [12] M. Fowler and K. Beck, *Refactoring: improving the design of existing code*. Addison-Wesley Professional, 2018.
- [13] M. Sewak, V. Emani, and A. Naresh, "Crush: Cybersecurity research using universal llms and semantic hypernetworks," in *EKG-LLM@CIKM*, 2023.
- [14] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (llm) security and privacy: The good, the bad, and the ugly," *High-Confidence Computing*, p. 100211, 2024.
- [15] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "On hardware security bug code fixes by prompting large language models," *IEEE Transactions on Information Forensics and Security*, pp. 4043–4057, 2024.
- [16] S. Kwon, S. Lee, T. Kim, D. Ryu, and J. Baik, "Exploring llm-based automated repairing of ansible script in edge-cloud infrastructures," *Journal of Web Engineering*, pp. 889–912, 2023.
- [17] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Assisting static analysis with large language models: A chatgpt experiment," in *Proc. ACM ESEC-FSE*, 2023, pp. 2107–2111.
- [18] P. Liu, C. Sun, Y. Zheng, X. Feng, C. Qin, Y. Wang, Z. Xu, Z. Li, P. Di, Y. Jiang, and L. Sun, "Llm-powered static binary taint analysis," *ACM Transactions on Software Engineering and Methodology*, 2025.
- [19] J. Yu, P. Liang, Y. Fu, A. Tahir, M. Shahin, C. Wang, and Y. Cai, "An insight into security code review with llms: Capabilities, obstacles and influential factors," *arXiv preprint arXiv:2401.16310*, 2024.
- [20] "aquasecurity/tfsec: Tfsec is now part of trivy," accessed February 1, 2025. [Online]. Available: <https://github.com/aquasecurity/tfsec>
- [21] "terraform-linters/tflint: A pluggable terraform linter," accessed February 1, 2025. [Online]. Available: <https://github.com/terraform-linters/tflint>
- [22] "An introduction to immutable infrastructure," accessed February 1, 2025. [Online]. Available: <https://www.oreilly.com/ideas/an-introduction-to-immutable-infrastructure>
- [23] C. Parnin, E. Helms, C. Atlee, H. Boughton, M. Ghattas, A. Glover, J. Holman, J. Micco, B. Murphy, T. Savor *et al.*, "The top 10 adages in continuous deployment," *IEEE Software*, pp. 86–95, 2017.
- [24] J. Humble and D. Farley, *Continuous delivery: reliable software releases through build, test, and deployment automation*. Addison-Wesley Professional, 2010.
- [25] R. Shambaugh, A. Weiss, and A. Guha, "Rehearsal: A configuration verification tool for puppet," in *Proc. ACM SIGPLAN*, 2016, pp. 416–430.
- [26] Y. Jiang and B. Adams, "Co-evolution of infrastructure and source code-an empirical study," in *Proc. MSR*, 2015, pp. 45–55.
- [27] D. Di Nucci, F. Palomba, D. A. Tamburri, A. Serebrenik, and A. De Lucia, "Detecting code smells using machine learning techniques: Are we there yet?" in *Proc. IEEE SANER*, 2018, pp. 612–621.
- [28] S. Reis, R. Abreu, M. d'Amorim, and D. Fortunato, "Leveraging practitioners' feedback to improve a security linter," in *Proc. IEEE/ACM ASE*, 2023, pp. 1–12.
- [29] E. van der Bent, J. Hage, J. Visser, and G. Gousios, "How good is your puppet? an empirically defined and validated quality model for puppet," in *Proc. IEEE SANER*, 2018, pp. 164–174.
- [30] R. Opdebeeck, A. Zerouali, and C. De Roover, "Smelly variables in ansible infrastructure code: detection, prevalence, and lifetime," in *Proc. MSR*, 2022, pp. 61–72.
- [31] N. Bessghaier, M. Begoug, C. Mebarki, A. Ouni, M. Sayagh, and M. W. Mkaouer, "On the prevalence, co-occurrence, and impact of infrastructure-as-code smells," in *Proc. IEEE SANER*, 2024, pp. 23–34.
- [32] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, pp. 1–36, 2023.
- [33] Y. Wu, Y. Zhang, T. Wang, and H. Wang, "Characterizing the occurrence of dockerfile smells in open-source software: An empirical study," *IEEE Access*, pp. 34 127–34 139, 2020.
- [34] N. Saavedra and J. a. F. Ferreira, "Glitch: Automated polyglot security smell detection in infrastructure as code," in *Proc. IEEE/ACM ASE*, 2023, pp. 1–12.
- [35] N. Saavedra, J. a. Gonçalves, M. Henriques, J. a. F. Ferreira, and A. Mendes, "Polyglot code smell detection for infrastructure as code with glitch," in *Proc. IEEE/ACM ASE*, 2024, pp. 2042–2045.
- [36] M. Begoug, M. Chouchen, A. Ouni, E. Abdullah Alomar, and M. W. Mkaouer, "Fine-grained just-in-time defect prediction at the block level in infrastructure-as-code (iac)," in *Proc. MSR*, 2024, pp. 100–112.
- [37] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, and T. Xu, "Large language models as configuration validators," in *Proc. IEEE/ACM ICSE*, 2024, pp. 204–216.
- [38] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "Genkubesecc: Llm-based kubernetes misconfiguration detection, localization, reasoning, and remediation," *arXiv preprint arXiv:2405.19954*, 2024.
- [39] Z. Ye, T. H. M. Le, and M. A. Babar, "Llmsecconfig: An llm-based approach for fixing software container misconfigurations," *arXiv preprint arXiv:2502.02009*, 2025.
- [40] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, "Using an llm to help with code understanding," in *Proc. IEEE/ACM ICSE*, 2024, pp. 1–13.
- [41] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang, "Clarifygpt: Empowering llm-based code generation with intention clarification," *arXiv preprint arXiv:2310.10996*, 2023.
- [42] J. Ye, X. Chen, N. Xu, C. Zu, Z. Shao, S. Liu, Y. Cui, Z. Zhou, C. Gong, Y. Shen *et al.*, "A comprehensive capability analysis of gpt-3 and gpt-3.5 series models," *arXiv preprint arXiv:2303.10420*, 2023.
- [43] R. Ishizue, K. Sakamoto, H. Washizaki, and Y. Fukazawa, "Improved program repair methods using refactoring with gpt models," in *Proc. ACM SIGCSE*, 2024, pp. 569–575.
- [44] M. M. Imran, P. Chatterjee, and K. Damevski, "Uncovering the causes of emotions in software developer communication using zero-shot llms," in *Proc. IEEE/ACM ICSE*, 2024, pp. 1–13.
- [45] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "Chatunitest: A framework for llm-based test generation," in *Proc. ACM FSE*, 2024, pp. 572–576.
- [46] J. Xu, R. Yang, Y. Huo, C. Zhang, and P. He, "Divlog: Log parsing with prompt enhanced in-context learning," in *Proc. IEEE/ACM ICSE*, 2024, pp. 1–12.
- [47] X. Yin, C. Ni, and S. Wang, "Multitask-based evaluation of open-source llm on software vulnerability," *IEEE Transactions on Software Engineering*, pp. 3071–3087, 2024.
- [48] "All the open source code in github now shared within bigquery: Analyze all the code! — by felipe hoffa — medium," accessed February 1, 2025. [Online]. Available: <https://hoffa.medium.com/github-on-bigquery-analyze-all-the-code-b3576fd2b150>
- [49] "Understanding github code search syntax - github docs," accessed February 1, 2025. [Online]. Available: <https://docs.github.com/en/search-github/github-code-search/understanding-github-code-search-syntax#language-qualifier>
- [50] O. Dabic, E. Aghajani, and G. Bavota, "Sampling projects in github for msr studies," in *Proc. MSR*, 2021, pp. 560–564.
- [51] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshvanyk, and G. Bavota, "Using pre-trained models to boost code review automation," in *Proc. IEEE/ACM ICSE*, 2022, pp. 2291–2302.
- [52] A. Mastropaolo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, and G. Bavota, "On the robustness of code generation

techniques: An empirical study on github copilot,” in *Proc. IEEE/ACM ICSE*, 2023, pp. 2149–2160.

- [53] M. Alfadel, D. E. Costa, and E. Shihab, “Empirical analysis of security vulnerabilities in python packages,” *Empirical Software Engineering*, p. 59, 2023.

APPENDIX

A. False Positive Validation in LLM Models for Detecting Terraform Code Smells

To assess the accuracy of LLM-based code smell detection, we manually verify the False Positive (FP) rate of detected code smells across six GitHub repositories.

Tool	False Positive Rate
<i>Tfsec</i>	0%
<i>Checkov</i>	0%
<i>Terrascan</i>	0%
GPT-3.5 Turbo	15.3%
GPT-4	40.0%
GPT-4o	43.3%

TABLE III
FALSE POSITIVE RATES OF TRADITIONAL LINTERS AND LLMs IN
DETECTING TERRAFORM CODE SMELLS.

Table III presents the FP rates observed for both traditional static analysis tools and LLM models. Traditional static analysis tools exhibit no FP, as their rule-based detection strictly adheres to predefined heuristics. In contrast, LLM-based approaches show higher FP rates mainly on *law_of_demeter* (27.27%), *empty_password* (24.24%) and *incomplete_conditional* (12.12%), indicating cases where multiple code smells were erroneously flagged within a single configuration. Among the LLM models, GPT-3.5 Turbo produces an acceptable FP rate for practical use. While some misclassifications occur, its detection performance remains balanced, capturing a broader range of code smells while avoiding excessive FP. However, GPT-4 and GPT-4o introduce significantly higher FP rates, making their outputs less reliable without additional post-processing. For production environments, stricter thresholds for false positives may be necessary. Future work could explore filtering mechanisms or confidence scoring to adapt LLM outputs to these contexts.

We hypothesize these false positives stem from LLMs’ limited understanding of Terraform’s context-specific patterns: misinterpreting legitimate nested structures as *law_of_demeter* violations, failing to recognize valid password implementations (e.g., flagging passwords defined via variables as potential issues without access to the variable definition files), and struggling to trace conditional logic distributed across multiple code blocks. These specific detection challenges represent important areas for future investigation to develop more context-aware constraints for LLM-based code smell detection in Infrastructure as Code.

B. Code Smell Definition

Terraform code smells are categorized by their primary impact on infrastructure as code.

Security:

- 1) Admin by Default (*admin_by_default*): Resource configurations or IAM roles automatically grant administrative privileges by default.
- 2) Empty Password (*empty_password*): Instances where password fields are configured with zero-length strings.
- 3) Hard-coded Secret (*hard_coded_secret*): Directly embedding authentication credentials within code rather than utilizing variable references or secure credential management solutions.
- 4) Insufficient Key Size (*insufficient_key_size*): Implementation of cryptographic keys smaller than security standards recommend, such as RSA keys below 2048 bits.
- 5) Invalid Port Range (*invalid_port_ranges*): Network port configurations that exceed the valid TCP range (0-65535).
- 6) Suspicious Comment (*suspicious_comment*): Comments containing references to credentials or functional defects.
- 7) Unrestricted IP Address (*unrestricted_ip_address*): Binding services to universal addresses like “0.0.0.0” or “:”, permitting connections from any source.
- 8) Weak Crypto Algorithm (*weak_crypto_algorithms*): Utilization of encryption methods known to have vulnerabilities or that have been deprecated.

Maintainability:

- 9) Complex Expression (*complex_expression*): Code segments containing expressions that create comprehension challenges.
- 10) Deprecated Statement Usage (*deprecated_statement_usage*): The configuration code uses one of the deprecated statements (such as `homepage` and `name` attributes in the `azuread_application` resource).
- 11) Incomplete Tasks (*incomplete_tasks*): Presence of “fixme” or “todo” annotations indicating pending implementation work.
- 12) Invalid Property Value (*invalid_property_value*): Incorrect parameter values, such as file permissions specified with 3-digit rather than 4-digit octal notation.
- 13) Law of Demeter (*law_of_demeter*): Configuration code violates the Law of Demeter when it accesses deeply nested attributes or module hierarchies instead of using direct references or explicit interfaces. This typically appears in three ways: deep attribute chaining (e.g., `resource.attribute.nested.value`), module traversal (e.g., `module.something.resource.internal.attribute`), and complex output references that expose internal structures instead of clean interfaces (e.g., `module.app.network_details.subnets.private[0].id`).

Robustness:

- 14) Incomplete Conditional (*incomplete_conditional*): Conditional expressions use chained ternary operators (`condition ? true_val : false_val`) without implementing an explicit default case that handles all possible input values.